

Radix Sort Of IEEE 754 Floating Point Numbers

Tomas Ukkonen

Novel Insight Research, Finland

Abstract

Radix sort is one of the fastest sorting algorithm but it only works with integers. In this paper, an order preserving isomorphism is described between IEEE 754 floating point numbers and little endian integers on x86_64 platform. This makes it possible to use the vanilla radix sort when a conversion code $O(n)$ between floating point numbers are integers is called before and after applying the radix sort algorithm. If the number of elements N is large enough, this is faster than the standard quicksort algorithm.

Keywords: radix sort, floating point numbers

1. Introduction

Radix sort is the basis for some of the fastest sorting algorithms [1, 2]. Therefore extending it to work with floating point numbers is very useful. In this paper, non-optimized text-book radix sort is used to sort floating point numbers and it is compared to C's standard quicksort. The speed-up results hold for IEEE 754 floating point numbers[3] which can be easily modified with order preserving isomorphism to integers of the same bitsize.

2. Order preserving isomorphism

Because the number of different values are same for 32-bit integers and 32-bit floating points and 64-bit integers and 64-bit floating points, it is always possible to map floating point values to integers and integers back to floating point values so that ordering of elements is not changed. To sort numbers, it is possible to convert floating point numbers to integers, sort them using the radix sort and convert integers back to floating point values.

The conversion code can be done efficiently for IEEE 754 floating point numbers (C++ code, Listing 1). Conversion handles NaNs and infinities as special cases and for normal numbers handles the sign bit and keeps a positive number's exponent and mantissa intact so they are in order in binary integer format and reverses exponent and mantissa if the number is negative.

The conversion code can be computed in parallel for each value meaning that conversion code $O(N)$ should have smaller and smaller effect on the execution time.

Email address: tomas.ukkonen@novelinsight.fi (Tomas Ukkonen)

URL: www.novelinsight.fi (Tomas Ukkonen)

3. Experimental results

To test the C++ non in-place radix sort for floating points, an experiment using uniform random numbers was run with different number of floating point values and compared to C language’s quicksort. Experiments were run on AMD Ryzen 7 7745HX processor (Linux) in a single thread 20 times and average was calculated. From Table 1 we can see that the floating point radix sort is faster when the number of values is larger than five million.

On the other hand, parallel processing of the conversion code using OpenMP didn’t have significant effect on the execution times and was slower with small values of N . This is probably because of the overhead causes by parallel processing.

Data Size	Quicksort	Floating point radix sort
100.000	0.01231	0.01978
500.000	0.06461	0.07726
1.000.000	0.1310	0.1516
5.000.000	0.7534	0.7204
10.000.000	1.529	1.312
100.000.000	17.26	11.36
200.000.000	35.75	22.08

Table 1: Execution time in seconds (single thread) for sorting N single precision floating point numbers

4. Discussion

The speed-up gained by using the radix sort is approximately 1.5 times faster execution speed and even faster results is possible with in-place optimized parallel radix sort. The speed-up holds only for large values of N but it still implies a potential optimization target for the binary format of floating point numbers. If the floating point’s binary format would be compatible with the radix sort, the sorting of most numbers could be done directly without asymptotically slower quicksort.

5. Funding and further information

This research has not received any funding or grants from any sources. The floating point radix sorting algorithm is published open source as a part of Dinrhiw2 C++ machine learning software library. [4].

Listing 1: Conversion code for converting floating point values to integers and back to floating point values

```
#include <string.h>

// maps IEEE 754 single precision floating point number to
// unsigned 32-bit integer
unsigned int ieee754spf2uint32(const float& f)
{
    unsigned int i;

    memcpy(&i, &f, sizeof(f));

    // checks if number is NaN or Inf
    if(((i >> 23) & 0xFF) == 0xFF){

        // checks lowest 23 bits (mantissa)
        if((i & 0x7FFFFFF) != 0){ // NaN
            // two forms of NaN: plus or minus
            if((i & 0x80000000) == 0){ // non-negative
                return 0xFFFFFFFF; // NaN are decided to be "biggest" values
            }
            else{ // sign bit on: negative
                return 0xFFFFFFF;
            }
        }
        else{ // infinities
            if((i & 0x80000000) == 0){ // non-negative
                return 0xFFFFFFF; // +Inf is biggest right after NaNs
            }
            else{ // sign bit on: negative
                return 0; // -Inf is smallest value
            }
        }
    }
    else if(((i >> 23) & 0xFF) == 0){ // zero exponent

        if((i & 0x7FFFFFF) == 0){ // zero
            if(i & 0x80000000){ // negative zero

                return 0x80000000; // sign bit on, zero exponent, zero mantissa
            }
            else{ // positive zero

                return 0x80000001; // sign bit on, zero exponent, mantissa = 1
            }
        }
    }

    // handles 'normal' number representation

    // toggles the sign bit
    i = i ^ 0x80000000;

    // if f is negative, reverses mantissa, reverses exponent
    if((i & 0x80000000) == 0){
        // reverse(exp)                                reverses mantissa
        i = (0x7F800000 - (i & 0x7F800000)) + (0x7FFFFFF - (i & 0x7FFFFFF));
    }
}
```

```

    }
    else{ // positive numbers
        // makes room for two zeros
        i += 2;
    }

    return i;
}

float uint322ieee754spf(const unsigned int& i) // inverse
{
    float f;
    unsigned int j;

    // checks for NaNs, Infs and zeros

    if(i == 0xFFFFFFFF){ // positive NaN
        j = 0x7FFFFFFF;
    }
    else if(i == 0xFFFFFFF0){ // negative NaN
        j = 0xFFFFFFF0;
    }
    else if(i == 0xFFFFFFF8){ // +Inf
        j = 0x7F800000;
    }
    else if(i == 0){ // -Inf
        j = 0xFF800000;
    }
    else if(i == 0x80000000){ // negative zero
        j = 0x80000000;
    }
    else if(i == 0x80000001){ // positive zero
        j = 0x0;
    }
    else{
        // 'normal' numbers

        if((i & 0x80000000) == 0){ // negative number

            // reverses exponent                reverses mantissa
            j = (0x7F800000 - (i & 0x7F800000)) + (0x7FFFFFFF - (i & 0x7FFFFFFF));

            j = j ^ 0x80000000; // toggles signbit
        }
        else{ // positive floating point number
            j = i - 2;

            j = j ^ 0x80000000; // changes signbit
        }
    }

    memcpy(&f, &j, sizeof(f));

    return f;
}

```

```

/*****/

// maps IEEE 754 double precision floating point to
// unsigned 64-bit integer
unsigned long long ieee754dpf2uint64(const double& f)
{
    unsigned long long i;
    memcpy(&i, &f, sizeof(f));

    // checks if number is NaN or Inf
    if((i >> 52) & 0x7FF) == 0x7FF){

        // checks lowest 52 bits (mantissa)
        if((i & 0xFFFFFFFFFFFFFFFFULL) != 0){ // NaN

            // two forms of NaN: plus or minus
            if((i & 0x8000000000000000ULL) == 0){ // non-negative
                // NaN are decided to be "biggest" values
                return 0xFFFFFFFFFFFFFFFFULL;
            }
            else{ // sign bit on: negative
                return 0xFFFFFFFFFFFFFFFFEULL;
            }
        }
        else{ // infinities
            if((i & 0x8000000000000000ULL) == 0){ // non-negative
                // +Inf is biggest right after NaNs
                return 0xFFFFFFFFFFFFFFFFDULL;
            }
            else{ // sign bit on: negative
                return 0ULL; // -Inf is smallest value
            }
        }
    }
    else if((i >> 52) & 0x7FF) == 0){ // zero exponent

        if((i & 0xFFFFFFFFFFFFFFFFULL) == 0){ // zero (mantissa is zero)
            if(i & 0x8000000000000000ULL){ // negative zero
                // sign bit on, zero exponent, zero mantissa
                return 0x8000000000000000ULL;
            }
            else{ // positive zero

                // sign bit on, zero exponent, mantissa = 1
                return 0x8000000000000001ULL;
            }
        }
    }

    // handles 'normal' number representation

    // toggles the sign bit
    i = i ^ 0x8000000000000000ULL;

    // if f is negative, reverses mantissa, reverses exponent

```

```

    if((i & 0x8000000000000000ULL) == 0){
        // reverse(exp)                                reverses mantissa
        i = (0x7FF0000000000000ULL - (i & 0x7FF0000000000000ULL)) +
            (0xFFFFFFFFFFFFFFFFULL - (i & 0xFFFFFFFFFFFFFFFFULL));
    }
    else{ // positive numbers
        // makes room for two zeros
        i += 2;
    }

    return i;
}

double uint642ieee754dpf(const unsigned long long& i) // inverse
{
    double f;
    unsigned long long j;

    // checks for NaNs, Infs and zeros

    if(i == 0xFFFFFFFFFFFFFFFFULL){ // positive NaN
        j = 0x7FFFFFFFFFFFFFFFFULL;

        // slow: optimize with assembly to
        // few simple&fast assembly commands
        memcpy(&f, &j, sizeof(f));
        return f;
    }
    else if(i == 0xFFFFFFFFFFFFFFFFEULL){ // negative NaN
        j = 0xFFFFFFFFFFFFFFFFULL;

        memcpy(&f, &j, sizeof(f));
        return f;
    }
    else if(i == 0xFFFFFFFFFFFFFFFFDULL){ // +Inf
        j = 0x7FF0000000000000ULL;

        memcpy(&f, &j, sizeof(f));
        return f;
    }
    else if(i == 0){ // -Inf
        j = 0xFFF0000000000000ULL;

        memcpy(&f, &j, sizeof(f));
        return f;
    }

    else if(i == 0x8000000000000000ULL){ // negative zero
        j = 0x8000000000000000ULL;

        memcpy(&f, &j, sizeof(f));
        return f;
    }
    else if(i == 0x8000000000000001ULL){ // positive zero
        j = 0x0ULL;

        memcpy(&f, &j, sizeof(f));

```

```

    return f;
}

// 'normal' numbers

if((i & 0x8000000000000000ULL) == 0){ // negative number

    // reverses exponent                reverses mantissa
    j = (0x7FF0000000000000ULL - (i & 0x7FF0000000000000ULL )) +
        (0xFFFFFFFFFFFFFFFFULL - (i & 0xFFFFFFFFFFFFFFFFULL));

    j = j ^ 0x8000000000000000ULL; // toggles signbit

    memcpy(&f, &j, sizeof(f));
    return f;
}
else{ // positive floating point number
    j = i - 2;

    j = j ^ 0x8000000000000000ULL; // toggles signbit

    memcpy(&f, &j, sizeof(f));
    return f;
}
}

```

References

- [1] M. Cho, D. Brand, R. Bordawekar, U. Finkler, V. Kulandaisamy, R. Puri, PARADIS: An efficient parallel algorithm for in-place radix sort, Proceedings of the VLDB Endowment, Volume 8, Issue 12 (2015). doi:10.14778/2824032.282405.
- [2] C. Kim, S. Yoon, D. Kim, Fast sort of floating-point data for data engineering, Advances in Engineering Software, Volume 42, Issues 1-2 (2011). doi:10.1016/j.advengsoft.2010.10.017.
- [3] IEEE standard for floating-point arithmetic, IEEE standard 754.
URL <https://webstore.ansi.org/standards/ieee/ieeestd7542008>
- [4] T. Ukkonen, Dinrhiw2: C++ machine learning library (2022).
URL <https://github.com/cslr/dinrhiw2>